

ContractScanner

Pay-per-scan Solidity security auditing, settled in USDC on Circle's Arc. Each scan is one metered, USDC-settled unit of compute.

Track 4 · Agentic Economy on Arc

Circle product: USDC

Chain 5042002 (Arc testnet)

Demo: www.gaziblockchain.tech Repo: github.com/0xrama-k/ContractScanner

Contact: ramazankaratas626@gmail.com

01 Title & short description

ContractScanner — pay-per-scan Solidity security auditing, settled in USDC on Arc.

ContractScanner is a pay-per-use AI security service. A developer pastes or uploads a Solidity contract and pays a small, fixed **USDC** fee on **Circle's Arc** testnet; once the on-chain payment confirms, the backend runs Slither static analysis inside a network-disabled Docker sandbox, normalizes and risk-scores the findings, optionally enriches them with an LLM explanation layer, and returns an actionable report (viewable in-app and exportable as JSON or Markdown). Each scan is one metered, USDC-settled unit of compute — a clean example of a real-time, pay-per-inference service running on programmable stablecoin rails.

02 Track & Circle products

Track

Track 4 — Best Agentic Economy Experience on Arc.

Direct fit with the track's "pay-per-inference AI agents that pay for each model response or dataset access in real time" example: each scan is metered and paid for in USDC at the moment of use, with analysis kicked off automatically the instant the payment settles on Arc.

Circle products used on Arc

✓ USDC	Native settlement rail; per-scan fee paid as native USDC value via <code>pay(bytes32)</code>
— Wallets	User-owned injected wallets (EIP-6963). Roadmap.
— Gateway / CCTP	Roadmap (multi-chain funding & treasury)
— Nanopayments	Roadmap (per-detector streaming pricing)

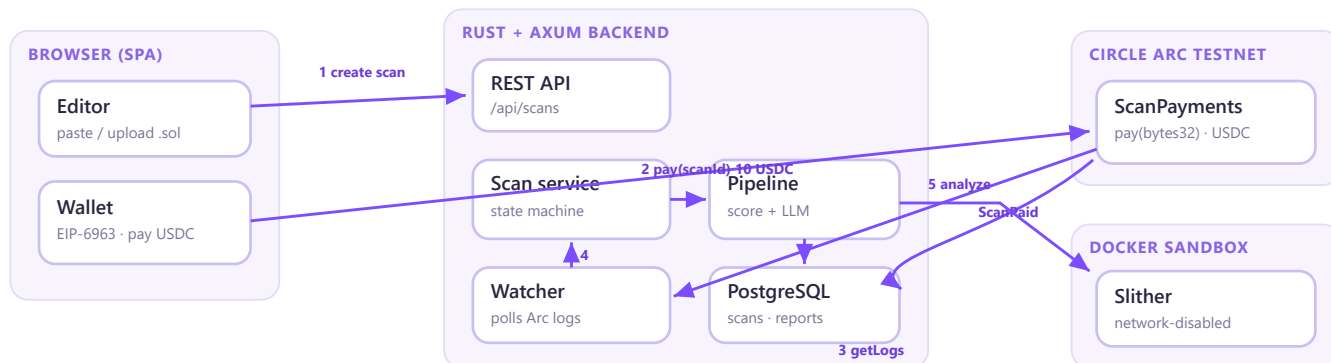
03 Why USDC on Arc

A security scan is a discrete, metered unit of compute — the textbook shape for a pay-per-use rail. Two Arc properties made it the right fit:

- **USDC is the native asset on Arc**, so the scan fee is collected directly through a payable function's `msg.value` — no ERC-20 approve round-trip, one wallet confirmation per scan, and an import-free contract.
- **Deterministic finality** lets the backend treat a confirmed `ScanPaid` event as settlement and immediately move the scan from `awaiting_payment` to `queued` — a genuine real-time "pay → scan starts" experience.

Decimals: on Arc, native USDC (`msg.value`) uses **18 decimals** while the USDC ERC-20 interface uses 6. The contract's $\text{PRICE} = 10 * 10^{**18}$ therefore means **10 USDC**.

04 Architecture



The frontend creates a scan (`awaiting_payment`) and gets a payment block (contract, `scanId` as `bytes32`, price, chain id). The wallet calls `pay(bytes32 scanId)` on Arc with the USDC fee as native value. The backend's watcher polls Arc for the `ScanPaid` event; on confirmation it moves the scan to `queued` and runs the pipeline. The page polls until `report_ready`. The frontend never asserts "I paid" — settlement is observed on-chain by the backend.

05 The payment contract

```
// contracts/ScanPayments.sol – self-contained, import-free
uint256 public constant PRICE = 10 * (10 ** 18); // 10 USDC (native, 18 dp on Arc)

function pay(bytes32 scanId) external payable {
    if (msg.value < PRICE) revert Underpaid();
    if (paid[scanId]) revert AlreadyPaid(); // replay protection
    paid[scanId] = true;
    emit ScanPaid(scanId, msg.sender, msg.value);
}
```

`scanId` is the backend's UUID left-padded to `bytes32`, binding each on-chain payment to exactly one off-chain scan. Price is immutable (no `setPrice`, no fee admin); the only privileged action is `withdraw` under two-step ownership. The backend trusts a payment only after N confirmations and re-verifies `amount ≥ price`.

06 Functional MVP & deliverables

Item	Where
Working frontend + backend	Rust/Axum + PostgreSQL + Dockerized Slither; single-page frontend with EIP-6963 wallet flow paying USDC on Arc
Architecture diagram	README.md & docs/ARCHITECTURE.md (Mermaid, GitHub-rendered) + diagram above
Video demonstration	drive.google.com/file/d/1p5cYC1xNct9HxHwnUOp86WR6lbvDfagt/view

Code repository	github.com/Oxrama-k/ContractScanner (setup + Circle integration in README)
Demo application	www.gaziblockchain.tech
Arc network	Chain 5042002 · RPC rpc.testnet.arc.network · Explorer testnet.arcscan.app

07 Circle Product Feedback

Why we chose these products

We chose **USDC on Arc** because a security scan is a discrete, metered unit of compute — the textbook shape for a pay-per-use rail. On Arc, **USDC is the native asset**, so the fee is collected directly through a `payable` function's `msg.value` (no `approve` + `transferFrom`), and **deterministic finality** lets us treat a confirmed `ScanPaid` event as settlement and start analysis instantly. Dollar-denominated fixed pricing (10 USDC/scan) keeps the UX legible for developers who are not crypto-native.

What worked well

- **Native USDC value transfer** collapsed our payment integration to a single `payable` call. Our original prototype targeted a native-gas-token chain, and migrating to Arc was nearly mechanical precisely because USDC behaves as the native asset — the contract logic did not change.
- **EVM compatibility** meant existing tooling (Foundry `forge create`, raw `eth_getLogs/eth_blockNumber` polling, EIP-6963 discovery, `wallet_addEthereumChain`) worked against Arc with only endpoint and chain-id changes.
- Clear, stable **contract addresses and network parameters** in the Arc docs made wiring up RPC, chain id, and explorer straightforward.

What could be improved

- **The 18-vs-6 decimal split** for native USDC vs. the ERC-20 interface is a sharp edge — easy to write an off-by- 10^{12} bug. First-class conversion helpers and louder warnings around `balanceOf` reading 0 for tiny native balances would help.
- **Faucet + onboarding discoverability**: because USDC doubles as gas, a new deployer needs testnet USDC before doing anything. A single “connect on Arc testnet” quickstart that funds gas would shorten first-run time.
- **Public-RPC `eth_getLogs` range limits** aren't documented in one obvious place; we kept a conservative 100-block window. Publishing exact limits per provider would help event-indexing backends.

Recommendations

- Ship a reference “**pay-per-use / metered service**” sample (`payable`-fee contract + event watcher) — a common agentic-economy pattern that maps directly onto Arc's native-USDC model.
- Provide official **decimal-conversion utilities** (native ↔ ERC-20) in the JS/Rust SDKs so builders never hand-roll the 10^{12} factor.
- Offer a **hosted event-indexer or contract-event webhooks** so backends don't poll `eth_getLogs` — our watcher would collapse to a webhook handler.

Roadmap (Circle products we'd add next)

- **Circle Wallets** — embedded wallets so non-crypto-native users can pay without a browser extension.
- **Circle Gateway / CCTP** — fund scans from USDC on other chains; route settlement for a hosted, multi-tenant version.
- **Nanopayments** — sub-cent, per-detector or streaming pricing instead of a flat per-scan fee.

